



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

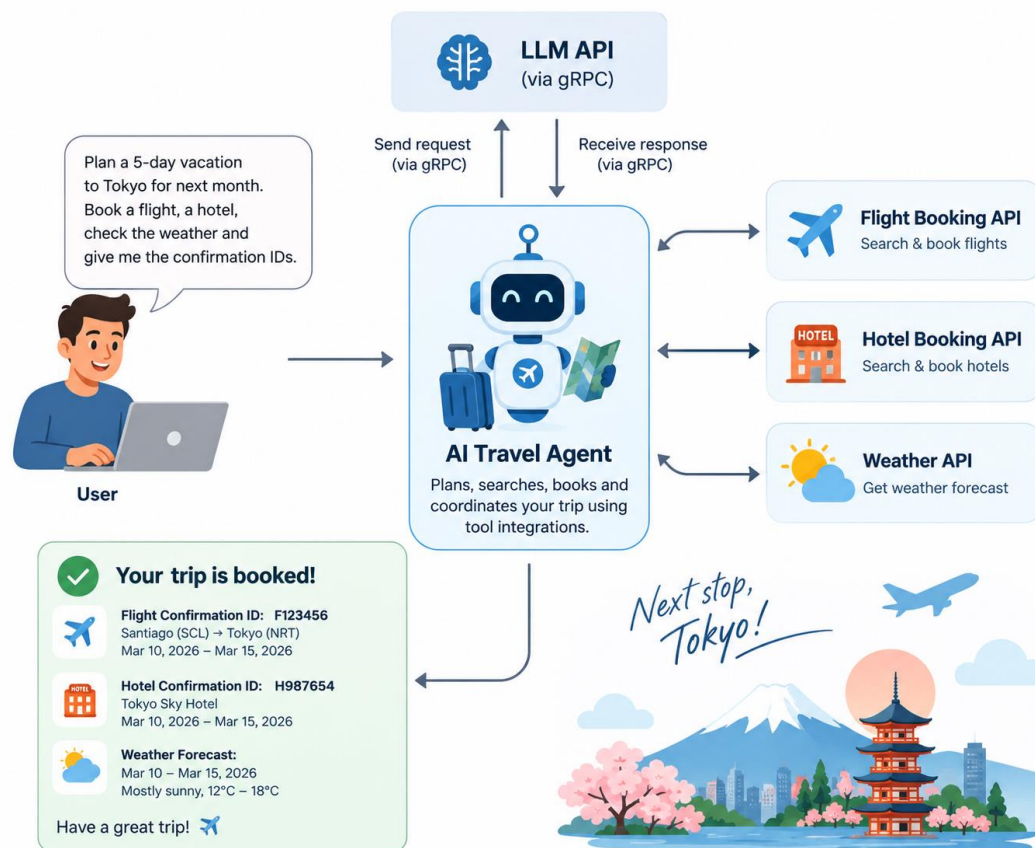
Enunciado Tarea 2

Objetivo

En esta tarea, desarrollarás un agente de IA para planificar tus vacaciones usando MCP tools.

Durante el proceso, aprenderás a:

- Interactuar con una API gRPC
- Interactuar con un LLM
- Implementar un multi-step agent loop.
- Resolver un caso de uso real (planificar vacaciones) usando IA.





IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Objetivo	1
Trabajo a realizar	2
Contexto.....	2
Servicios (MCPs) disponibles	2
Desafío	4
Comportamiento de la aplicación	5
1. Tarea 1	5
2. Tarea 2	5
Metodología e instrucciones de implementación	5
Arquitectura de la aplicación	6
API LLM (gRPC).....	6
Loop de agente	7
Base de datos.....	7
Despliegue	8
Informe de Aprendizajes.....	8
Instrucciones adicionales.....	8
Bibliotecas permitidas y prohibidas.....	8
Versionamiento de código	9
Entregables	9
Fecha de entrega	9
Requisitos mínimos.....	9
Penalizaciones.....	9
Uso de IA	10
Versiones del documento	10
Anexos.....	11
Documentación.....	12
Rúbrica Tentativa	12



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Trabajo a realizar

Contexto

Eres estudiante de **IIC3103 – Taller de Integración**, durante el semestre 2026-2. De pronto, se te ocurre una idea que podría cambiar tu futuro para siempre: emprender y crear **IntegraTrip**, un servicio para ayudar a las personas a planificar las vacaciones de sus sueños.

Planificar un viaje suele ser un proceso **manual, lento y fragmentado**. Para organizar unas vacaciones, una persona debe buscar vuelos, revisar opciones de alojamiento, consultar el clima y comparar información proveniente de distintas plataformas.

Ya construiste la infraestructura necesaria para que el proyecto funcione. Solo falta integrar IA generativa para poder llamar a los distintos MCPs que un usuario tiene conectado en tu plataforma.

Servicios (MCPs) disponibles

Para construir IntegraTrip, identificas tres servicios que pueden ser utilizados para planificar un viaje:

[Andes Air - Aerolínea:](#) Servicio que permite consultar información sobre vuelos.

The screenshot shows the Andes Air website with a dark blue header. The logo 'ANDES AIR' is on the left, and 'IIC3103 · MCP PRE' is in the center. On the right, 'Authoriz' is partially visible. The main content area has a background image of a sunset over a body of water. Large white text reads 'SANTIAGO AL MUNDO EN UNA ESCALA.' Below this, smaller text says 'Busca el catálogo determinista del curso. Misma fecha, mismo vuelo. Entra con tu email UC.' At the bottom, there is a search form with four fields: 'ORIGEN' (Santiago (SCL)), 'DESTINO' (Cancun (CUN)), 'FECHA' (31-08-2026), and 'PAX' (1). A red 'Buscar' button is on the right.

ORIGEN	DESTINO	FECHA	PAX
Santiago (SCL)	▼ Cancun (CUN)	▼ 31-08-2026	1

Buscar

[StayWell - Hoteles:](#) Servicio que permite consultar alternativas de alojamiento.



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Staywell.cl Authorization Server

Encuentra tu próxima estadía

Catálogo IIC3103 · DCR. Entra, busca, reserva y cancela — las mismas tools que tu cliente MCP.

Destino
Cancun (CUN)

Check-in
31-08-2026

Check-out
04-09-2026

Huéspedes
2

Buscar

Tus reservas

Costa Pacífico Cancun
CUN · 2026-08-31 → 2026-09-04 · 2 huéspedes · USD 936
BK-2b8f664c · confirmed

Cancelar

Andes Plaza Cancun
CUN · 2026-08-31 → 2026-09-04 · 2 huéspedes · USD 1012
BK-d0234cdf · confirmed

Cancelar

[Cielo Sur - Clima](#): servicio que permite consultar información meteorológica.

Cielo Sur

Ciudad
Cancun (CUN)

Desde
17-08-2026

Días
5

CANCUN · CUN · 2026-08-17

30°

Soleado. Mín 22° · tropical

08-17

30°

Soleado

08-18

29°

Cielo limpio

08-19

26°

Parcialmente nublado

08-20

19°

Parcialmente nublado

Estos servicios exponen sus funcionalidades utilizando **Model Context Protocol (MCP)**. Cada servicio soporta un tipo de autenticación distinta. Concretamente:

Andes Air: Pre-Registered OAuth Client (PRE): OAuth clásico, donde el cliente debe registrarse previamente para hacer el flujo de OAuth.

StayWell: Dynamic Client Registration (DCR): Permite registrar clientes dinámicamente, haciendo un request de registro durante el flujo de OAuth.



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Cielo Sur: Client ID Metadata Document (CMID): El cliente expone metadata mediante una URL, que es usada como client id para que el server pueda consultar estos detalles y crear el cliente OAuth.

Desafío

Incorporar un LLM para interactuar con los MCPs previamente conectados a tu plataforma.

Tu aplicación deberá:

- Todo lo de la tarea 1. Dejar en una sección o apartado de configuración.
- Implementar una vista de chat donde:
 - El usuario podrá iniciar un nuevo chat
 - El usuario podrá ver y retomar sus chats anteriores
 - El usuario podrá hablar en el chat con el agente para poder planificar sus vacaciones
 - El agente deberá pedir toda la información necesaria y pedir una confirmación antes de reservar los vuelos y el hotel.

Comportamiento de la aplicación

A grandes rasgos, la aplicación debe permitir implementar los siguientes casos de uso:

1. Tarea 1

Permitir a un usuario iniciar sesión, conectar MCPs que usan distintas variaciones de OAuth, y listar estas conexiones de forma persistente y segura.

2. Tarea 2

En esta tarea, se usa como base lo implementado en la Tarea 1. Todo esto quedará bajo una sección o apartado de configuración, donde los usuarios podrán seguir usando esta sección al igual que antes. Además:

1. Mantener login + conexión a **los tres** MCP (Pre, DCR, CIMD) en una vista de **Configuración**.
2. Vista de **Chat** a pantalla completa, distinta de la de configuración (navegación clara entre ambas).
3. Armar el catálogo de tools conectadas (nombre, descripción, inputSchema, servidor).
4. Cliente gRPC de `iic3103.llm.v1.Llm/Generate` con metadata `x-student-email` y `x-student-id`. El .proto se baja de la landing del servidor de LLM.
5. Loop de agente (máximo **12** turnos por mensaje). Follow-ups reusan el historial USER / MODEL / TOOL.



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

6. Recomendado: Prefijar nombres de tools para que sean únicos y válidos (pre_search_flights, dcr_book_hotel, cimd_get_forecast, ...).
7. Ejecutar cada function_call con tools/call en el MCP correcto.
8. En el hilo: traza visible de cada tool (nombre, argumentos, resultado o error) y texto del agente con renderización de markdown (negrita, listas, código).
9. Historial de conversaciones **por usuario**: listar, abrir/retomar (mismo contexto gRPC), **nueva conversación** con contexto vacío.
10. Manejar rate limit (RESOURCE_EXHAUSTED) sin un busy-loop.

Metodología e instrucciones de implementación

Arquitectura de la aplicación

Misma arquitectura de su solución actual, a excepción de que lo hayan hecho solo con código de cliente (Frontend). Para esta tarea necesitarán un BFF o backend para poder interactuar con la API gRPC de la LLM, ya que gRPC no está soportado en Browsers (sin extensiones).

API LLM (gRPC)

La API de LLM que deberán usar en esta tarea es la siguiente:

<https://iic3103-tarea2-llm-z2fqxmm2ja-uc.a.run.app/>

IIC3103 · TAREA 2

Proxy LLM (gRPC)

Un turno del modelo por RPC. Ustedes implementan el loop del agente (tools MCP → Generate otra vez). No hay REST de generación.

TARGET GRPC

dns:///iic3103-tarea2-llm-z2fqxmm2ja-uc.a.run.app:443

Copiar

TLS

true

Descargar llm.proto

Ver proto

/health

1. Metadata (obligatoria)

Misma lista del curso que Tarea 1. Sin API key de Gemini.

x-student-email: you@uc.cl
x-student-id: 12345678

2. Generar stubs desde este proto

El contrato vive en este servidor. Bájalo y genera las libs en tu repo.

Python

curl -fsSL https://iic3103-tarea2-llm-z2fqxmm2ja-uc.a.run.app/proto/llm.proto -o llm.proto
python3 -m pip install grpcio grpcio-tools
python3 -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. llm.proto

Node

llm.proto

Servicio iic3103.llm.v1.LLM RPC unary Generate

syntax = "proto3";

package iic3103.llm.v1;

// Course LLM proxy for IIC3103 Tarea 2.
// Each Generate call is a single model turn. Your ap
// send tools + history, execute any function_calls a
// messages, call Generate again until text is set (o

service LLM {
 rpc Generate(GenerateRequest) returns (GenerateResp
 }

message Tool {
 // Unique name: [A-Za-z_][A-Za-z0-9_]{0,63}
 // MCP names collide across servers (e.g. list_book
 // e.g. pre_search_flights, dcr_book_hotel, cimd_ge
 string name = 1;



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

En esta podrán revisar los [archivos .proto](#) que indican exactamente el protocolo que espera la API, y les permitirán generar el código necesario para usarla (Ejemplos de Python y Node en la landing de la API).

Para la autenticación de la API deberán usar los header: x-student-email, x-student-id. Indicando su email y número de alumno. (este puede quedar fijo en su backend no es necesario usar las credenciales del usuario loggeado en su app)

Códigos de respuesta útiles:

gRPC	Cuándo
UNAUTHENTICATED	Falta metadata
PERMISSION_DENIED	Email / número no están en la lista
RESOURCE_EXHAUSTED	Rate limit (por defecto 12 llamadas / 60 s por estudiante)
INVALID_ARGUMENT	Request inválido (nombres, tamaño, JSON)
INTERNAL	Error de Gemini

Loop de agente

A grandes rasgos, su loop de agente debería verse de la siguiente forma (Pseudocódigo):

```
catalog = tools de los MCP conectados, con nombres únicos
messages = historial de este chat (vacío si es conversación nueva)
messages += USER(texto del recuadro)
for turn in 1..12:
    resp = Generate(messages, tools)
    if resp.function_calls:
        ejecutar cada una con tools/call en el mcpUrl mapeado
        messages += MODEL(function_calls) + TOOL(function_results)
        continue
    mostrar resp.text, guardar messages, terminar
```

Si el modelo pide varias tools en el mismo turno, ejecútenlas todas antes del siguiente **Generate**.



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Un follow-up (Ej. “sí, reserva esos”) **reusa** `messages`. “Nueva conversación” **no** manda ese historial.

Si reciben `RESOURCE_EXHAUSTED`, muestren el error y **no reintenten en un bucle cerrado. Es decir, no eperen a nivel de backend para reintentar. El usuario debería esperar y reintentar enviando un nuevo mensaje desde el chat.**

Base de datos

Además de lo implementado en la **Tarea 1** (sesión y MCPs conectados), en esta tarea deberán guardar toda la información necesaria para implementar la **interfaz de chat**.

Esto implica:

- Guardar los **chats de cada usuario**.
- Guardar **todos los mensajes** de cada chat.
- Registrar las **llamadas a las tools** realizadas durante un mensaje, junto con la información necesaria para reconstruir la conversación.

Para esto, deberán **extender su modelo de base de datos** para soportar esta nueva información.

Despliegue

Informe de Aprendizajes.

Como parte de la tarea, deberán incluir un **informe de reflexión** sobre los aprendizajes obtenidos durante el desarrollo de este mini proyecto, considerando tanto lo trabajado en la **Tarea 1** como en la **Tarea 2**.

En el informe, deberán comentar, entre otros aspectos:

- Qué aprendieron durante el desarrollo del proyecto.
- Qué aspectos de la tarea les gustaron y cuáles no.
- Qué dificultades encontraron y cómo las resolvieron.
- Qué conceptos o tecnologías les resultaron más interesantes.
- Cualquier otro comentario, opinión o aprendizaje que quieran compartir sobre la experiencia.

La idea es que sea un espacio para **reflexionar sobre la experiencia de desarrollo**, más allá de simplemente describir lo que implementaron.



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Instrucciones adicionales

Bibliotecas permitidas y prohibidas

Permitido

- Lo ya permitido en Tarea 1
- Cliente gRPC: @grpc/grpc-js + @grpc/proto-loader, o grpcio / grpcio-tools en Python
- Cargar el .proto en runtime o generar stubs (el hosteado o el de este repo)
- Renderizar markdown en el cliente (sanitizado; no innerHTML crudo con texto del modelo sin escape)

Prohibido

- LangChain, CrewAI, Google ADK, OpenAI Agents SDK, u otro framework que implemente el loop por ustedes
- Llamar a Gemini, OpenAI u otro LLM distinto del proxy del curso
- Mandar access tokens MCP (u otros secretos) en Generate
- Usar un endpoint REST de generación en lugar de gRPC

Versionamiento de código

Todo el código de su servicio deberá estar versionado en un repositorio privado que será provisto por el equipo docente. Para desarrollar puedes trabajar en un repositorio propio mientras no hayamos dado este acceso al repositorio final.

Entregables

Cada estudiante deberá entregar:

- URL de su servicio desplegado y disponible públicamente en internet.
- Documento de aprendizajes.
- URI de su repositorio en github.

Esta información se deberá enviar antes de la fecha de entrega a través del buzón que se habilitará en Canvas.

Fecha de entrega

La tarea debe ser entregada a más tardar el día viernes 2 de Octubre, a las 18:00, y deberá quedar desplegada y disponible durante el periodo de corrección (2 semanas).

Requisitos mínimos

Las tareas que no cumplan con las siguientes condiciones no serán corregidas y serán evaluadas con la nota mínima:



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

- El código deberá estar versionado en su totalidad en el repositorio de GitHub Classroom de esta tarea.
- El servicio debe reflejar fielmente el código entregado en el repositorio.
- El servicio debe tener persistencia de los datos al menos durante el periodo de corrección.

Penalizaciones

Se descontarán 0,2 puntos de la nota de la tarea por cada hora de atraso en la entrega, contados a partir de la fecha estipulada en el punto anterior.

En caso de no poder desplegar la API, se evaluará con nota máxima 5,0.

Cualquier intento de copia, plagio o acto deshonesto en el desarrollo de la tarea, será penalizado con nota 1,1 de acuerdo con la política de integridad académica del DCC.

Se descontarán 0,5 puntos de la nota ante cualquier falta grave de seguridad, detalladas en la sección [Seguridad de credenciales y secretos](#)

Uso de IA

Se permite el uso de IA y agentes para la generación del código requerido en esta tarea. Se deberá versionar en el repositorio de la tarea, todas las conversaciones sostenidas con el agente de forma de validar que el desarrollo realizado es un proyecto original y no copia.

Que el estudiante no entienda lo que entrega o que su entrega incluya información falsa (alucinada) será considerado como una falta a la integridad académica.

No se permite el uso de IA en la generación del documento de arquitectura de solución requerido en esta tarea.



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Versiones del documento

- **Versión 1:** 6/09/2026
 - Publicación del enunciado



IIC3103 - Taller de Integración

Departamento Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica

Anexos

Documentación

- Qué es OAuth <https://auth0.com/es/intro-to-iam/what-is-oauth-2>
- Protocolo MCP <https://modelcontextprotocol.info>
- Documentación CIMD: <https://client.dev/>

Rúbrica Tentativa

Criterio	Descripción	Ponderación	No cumple	Bueno	Excelente
Despliegue de app		10%			
Informe de aprendizajes		10 %			
Vista de Chats		10 %			
Loop de agente		20 %			
Manejo de contexto entre conversaciones		20 %			
Visualización de mensajes y tools		10 %			
Respuestas tienen sentido y el objetivo se logra		20 %			